

Pro task 4 (QUEUED — do not start until task 3 / #158 is merged)

This file is a queue slot, not an active assignment. The active task is always `pro-task.md`. When task 3 lands, the orchestrator promotes this file's contents into `pro-task.md` and resets `pro-result.md`. Do not act on this file while `pro-task.md` still says task 3.

Leptos upgrade reconnaissance — READ-ONLY, no upgrade

- **Branch to create:** `worker/pro/leptos-upgrade-recon`
- **Effort:** medium. Research and inventory, not implementation.
- **Deliverable:** one document. **No dependency changes, no code changes.**

Why this task exists

`cargo audit` (added by your task 2) reports three advisories, all `unmaintained` rather than `vulnerabilities`:

Advisory crate	Comes from
<code>paste</code>	<code>leptos_dom 0.6.15</code> , <code>leptos_reactive 0.6.15</code>
<code>proc-macro-error</code>	<code>rhtml</code> → <code>leptos_hot_reload</code> → <code>leptos_macro 0.6.15</code>
<code>proc-macro-error2</code>	<code>leptos_macro 0.6.15</code>

All three trace to `leptos 0.6.15`. They are registered in `docs/DEPENDENCY_EXCEPTIONS.md` with a 90-day expiry, so the build will fail again when that lapses and force a decision.

The apparent fix is "upgrade leptos". Current is `0.6`; latest published is `0.9.0-beta`. But the `0.6` → `0.7` boundary was leptos's reactive-system rewrite, and this frontend is 58 files / ~19,000 lines. Nobody should commit to that without real numbers. **Your job is to produce the numbers, not to do the upgrade.**

Questions you must answer, with evidence

1. **Does upgrading actually remove the three advisories?** This is the first question and it may invalidate the whole premise. Check the dependency trees of leptos `0.7`, `0.8` and `0.9-beta` — do they still pull `paste`, `proc-macro-error`, `proc-macro-error2`? If the newest leptos still depends on them, the upgrade does not fix the advisories and the exception register is the correct long-term answer. **Say so plainly if that is what you find** — a negative result here is the most valuable outcome of this task, because it saves a rewrite.

2. **Which version should be the target?** 0.7, 0.8, or 0.9-beta. Consider that 0.9 is beta and this project is pre-V1. State a recommendation with reasons, including whether the project should wait for 0.9 stable.
3. **What actually breaks?** Read the official leptos migration guides for each boundary crossed. Then inventory THIS codebase: which of the 58 files under `crates/git-vista/src/` use APIs that changed? Produce a table — file, what it uses, what it becomes, rough difficulty.
4. **How much does M1.11's architecture help?** M1.11 (ADR 0024) split frontend state into framework-free `core.rs` files with no leptos dependency and thin `wasm-only` `signals.rs` glue. Quantify it: how many lines live in cores that a leptos bump cannot touch, versus glue that it rewrites? If the split holds up, that is a strong argument the upgrade is more tractable than the raw file count suggests — and a real vindication of that refactor. If it does not hold up, say that too.
5. **Does trunk / the wasm toolchain need anything?** Check whether the target leptos requires a different trunk version, `wasm-bindgen`, or `build config`.
6. **Scope estimate**, in the same terms this project uses elsewhere: an S/M/L/XL size, what it would touch, and the main risks.

How to investigate without changing anything

Do this in a **throwaway directory outside the repo** (`mktemp -d`), never in the worktree:
- `cargo add leptos@0.7` in a scratch crate to inspect resolved trees, or use `cargo tree` against a scratch manifest.
- `cargo search`, `docs.rs`, and the leptos GitHub release notes / migration guides for each boundary.

The worktree itself must end **clean apart from your one new document**. Do not run `cargo add`, do not edit `Cargo.toml`, do not touch `Cargo.lock`.

Allowed files

- `design-docs/2026-07-XX-leptos-upgrade-recon.md` (create — use the real date). `design-docs/` is gitignored, so this is a local working document, which is the right home for a scope study. Render a PDF twin beside it per the house rule for design docs.

That is the only file you create. Nothing else.

Forbidden

- **Any change to `Cargo.toml`, `Cargo.lock`, or any dependency.** This is a read-only study. An accidental lockfile change here would be a real problem.
- Anything under `crates/`.
- `docs/adr/**` — if you think the outcome deserves an ADR, recommend it in the document and let Max and Tom decide.
- `main`, other branches, force-push, branch deletion.

Acceptance criteria

- [] Question 1 answered with actual dependency-tree evidence, not inference.

- [] A target version recommended, with reasons.
- [] A per-file inventory of what breaks.
- [] The core-vs-glue split quantified in lines.
- [] Toolchain implications checked.
- [] An S/M/L/XL estimate with risks.
- [] `git status` in the worktree shows only the new document (plus its PDF).
- [] `Cargo.lock` is **unmodified** — verify with `git status` and say so.
- [] PR opened against `main`? **No.** `design-docs/` is gitignored; there is nothing to merge.
Just report the document's path in `pro-result.md`.

Hard rules

Unchanged from previous tasks: stay in `/home/tom/projects/Git-Vista-pro`, never touch host port 8080, never delete a branch, never force-push, commits as `claude_2010` with the noreply email, sign artifacts `thomas2010` with a real ISO timestamp.

Do not let this task drift into doing the upgrade. If you find yourself editing `Cargo.toml`, stop — that is a different task that has not been approved.

Signed: thomas2025 · 2026-07-27T21:45:00-04:00